AIMERZ.ai
Aim.Act.Achieve

NOTES

Interfaces in Java - I

Abstraction

Introduction to an Interface

Abstraction Using Abstract Classes

What is Abstraction?

Abstraction is a programming concept that helps us focus on essential features of an object or system while hiding the non-essential details. It's like looking at a car and seeing only the steering wheel, accelerator, and brakes, without worrying about the engine, transmission, and other internal components.

Real-World Analogy: Food Restaurant

Imagine you're ordering food at a restaurant. You don't need to know how the chef prepares the dish, what ingredients they use, or how the kitchen is organized. You only care about the food being served to you. This is abstraction in action! You're focusing on the essential feature (the food) while ignoring the non-essential details (the cooking process).

What is an Abstract Class?

An **abstract class** in Java is a class that **cannot be instantiated directly**. It provides a template for other classes but leaves certain methods to be defined by subclasses. Abstract classes can have both **abstract methods** (without a body) and **concrete methods** (with a body).

Real-World Analogy: Blueprints and Buildings:

Think of an abstract class as an **architect's blueprint**. The blueprint has guidelines for building but doesn't specify details like wall color or furniture type. Different contractors use the blueprint to build unique houses based on it.

What are Abstract Methods

An abstract method is a method declared in an abstract class that has no implementation. It's like a placeholder that says, "Hey, you need to implement this method in your own way!"

Real-World Analogy:

Imagine you're building a house using the blueprint. The blueprint might specify that you need to install a door, but it doesn't tell you how to install it. You need to figure that out yourself. Similarly, an abstract method is like

a specification that says, "You need to implement this method, but I won't tell you how."

Types of Methods in an Abstract Class

Abstract Methods:

Declared with abstract keyword.

No body, only a declaration.

Example:

```Java
abstract void sound();
```

Concrete Methods:

Have a complete implementation.

Example:

```Java
void sleep() {
      System.out.println("Sleeping");
}
```

Example Code of Abstract Class and Abstract Method

```Java
abstract class Animal {  // Abstract class
    abstract void sound();  // Abstract method (no body)

    void sleep() {          // Concrete method (with body)
        System.out.println("The animal is sleeping.");
    }
}
```

Why Use Abstract Classes?

Structure: Abstract classes provide a foundation or blueprint that can be expanded upon.

Reusability: Shared methods can be defined in the abstract class and reused in subclasses.

Flexibility: Subclasses can override or add unique methods.

Real-World Example of Abstraction

Example 1: Animal Sound System

Imagine we have different animals that make different sounds. An **Animal** abstract class can define a general structure, while subclasses (like **Dog** and **Cat**) provide specific implementations for each sound.

Example

```Java
abstract class Animal {
    // Abstract method - does not have a body
    abstract void sound();

    // Concrete method - has a body
    void sleep() {
        System.out.println("The animal is sleeping.");
    }
}

// Subclass for Dog
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

// Subclass for Cat
class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}
```

```java
public class Main {
    public static void main(String[] args) {
        Animal dog = new Dog();
        dog.sound();    // Outputs: Dog barks
        dog.sleep();    // Outputs: The animal is sleeping

        Animal cat = new Cat();
        cat.sound();    // Outputs: Cat meows
        cat.sleep();    // Outputs: The animal is sleeping
    }
}
```

Explanation

Abstract Method: sound() is defined as an abstract method in Animal, and each subclass provides its own implementation.

Concrete Method: sleep() is a regular method in Animal, so all animals will share the same sleep behavior.

Advantages and Disadvantages of Using Abstract Classes

Advantages

- **Provides Structure**: Gives a common template for subclasses, ensuring they all follow a particular design.
- **Partial Implementation**: Allows sharing common code across subclasses, reducing redundancy.
- **Enhances Flexibility**: Subclasses are free to define abstract methods as they need.

Disadvantages

- **Single Inheritance**: A class can only extend one abstract class, which limits flexibility in some cases.
- **Partial Abstraction**: Abstract classes don't fully support abstraction as interfaces do, as they can have concrete methods.

Real-Time Usage: Shape Drawing System

Consider a drawing application that needs different shapes like **Circle** and **Rectangle**. Each shape should have a way to calculate its **area** but may need different formulas.

Code Implementation

```Java
abstract class Shape {
    abstract void draw(); // Abstract method

    void description() {  // Concrete method
        System.out.println("This is a shape.");
    }
}

// Circle subclass
class Circle extends Shape {
    void draw() {
        System.out.println("Drawing a circle.");
    }
}

// Rectangle subclass
class Rectangle extends Shape {
    void draw() {
        System.out.println("Drawing a rectangle.");
    }
}

public class DrawingApp {
    public static void main(String[] args) {
        Shape circle = new Circle();
        Shape rectangle = new Rectangle();

            circle.draw();           // Outputs: Drawing a
circle.
            rectangle.draw();        // Outputs: Drawing a
rectangle.
```

```
            circle.description();  //  Outputs:  This  is  a
shape.
    }
}
```

Explanation

In this system:

Abstract Class: Shape provides a structure with draw() as an abstract method.

Concrete Methods: The description() method is defined in Shape, so it's shared among all shapes.

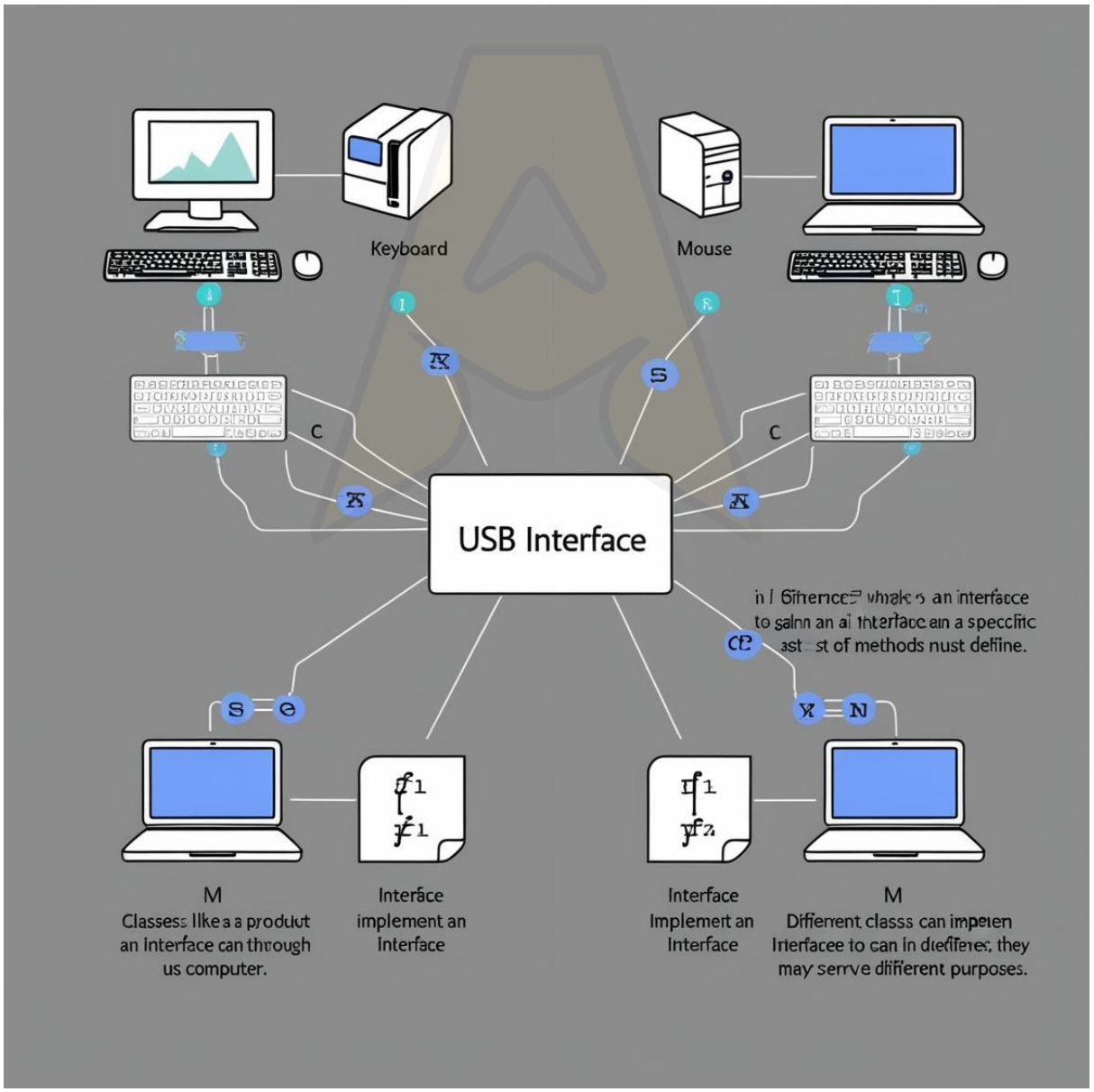
Result: This allows us to create different shapes with their own draw() methods while keeping common descriptions.

Introduction to an Interface

An interface in Java is a collection of abstract methods that a class can implement. It provides a way to **define behavior** that different classes can adopt, making sure in a consistent way for multiple classes to follow the same behavior without being related by inheritance.

Real World Analogy: USB

Imagine a USB interface. Different devices like a keyboard, mouse, or printer can connect to a computer through the USB port even though each device performs different functions. Similarly, in Java, different classes can implement an interface to gain a specific set of methods they must define, even though they may serve different purposes.



Real-World Example:
Let's say we have an interface called Movable for objects that can move, such as cars, bicycles, and animals. Each class that implements Movable must define how it moves, even though each moves differently.

```Java
interface Movable {
    void move();  // abstract method
}

class Car implements Movable {
    public void move() {
        System.out.println("The car drives on roads.");
    }
}

class Bicycle implements Movable {
    public void move() {
        System.out.println("The bicycle pedals on paths.");
    }
}

class Dog implements Movable {
    public void move() {
        System.out.println("The dog runs on grass.");
    }
}
```

Let's see, Key features of an interfaces;

- ➔ All methods by default are abstract and public.
- ➔ All variables are public static and final.
- ➔ Interfaces cannot be instantiated (Object cannot be created)
- ➔ A class must implement the interface and override all abstract methods to concrete or be declared abstract
- ➔ We can create object of the class that implements the interface and use the methods implemented

Let's come to an Important conclusion.

- ➔ One interface can be implemented by many classes
- ➔ Interface type ref can be used to point to implementing classes object to achieve loose coupling
- ➔ Interface type ref can be downcasted as well
- ➔ Interfaces can exhibit inheritance, single level,multi level, hierarchical, multiple but not cyclic
- ➔ One class can implement multiple interfaces at a time.
- ➔ A class can implement an interface and at the same time can extend from other classes.
- ➔ Extends keyword can exist between 2 class or 2 interfaces , but not between a class and interface, only 'implements' can exist between a class and interface.
- ➔ If a class "A" already has an implementation for a method "m1", that class B wants to implement for Itr1 then class B extending class A should suffice as an implementation for Itr1
- ➔ An interface which is empty is called a marker interface.
- ➔ An interface with one abstract method is called a Functional interface.

THANK YOU

AIMERZ.ai

Aim.Act.Achieve

Stay updated with us!

Vishwa Mohan

Vishwa Mohan

vishwa.mohan.singh

Vishwa Mohan